C# PRACTICAL ASSIGNMENT

CHAPTER 2 — METHODS, COLLECTIONS & LINQ

Instructions

1. Create a separate C# Console Application for each question.
2. Use user-defined methods wherever applicable.
3. Accept input from the user wherever required.
4. Display clear and meaningful output.

PART 1: METHODS AND BUILT-IN METHODS

1. Distance Converter

Create a program to convert distances between kilometres, metres and centimetres.

• Accept a distance and its unit from the user.

• Create separate methods for each conversion.

• Pass the distance as a parameter.

• Return the converted value.

• Display the result.

2. Text Formatter

Create a program that accepts a sentence from the user.

• Remove unnecessary spaces from the beginning and end.

• Convert the sentence into uppercase and lowercase.

• Replace a particular word with another word.

• Check whether the sentence contains a particular word.

• Check whether the sentence starts or ends with a particular word.

• Use appropriate String methods.

3. Mathematical Operations

Create a program that accepts two numbers.

• Find the absolute difference between the numbers.

• Find the maximum and minimum numbers.

• Calculate the square root of both numbers.

• Calculate the first number raised to the power of the second number.

• Round a decimal number entered by the user.

• Use Math methods.

4. Username Generator

Accept a person's first name, last name and birth year.

• Create a method to generate a username.

• Use string methods to format the names.

• Combine the first name, last name and birth year.

• Return the generated username.

• Display the result.

5. Date Planner

Create a program that displays upcoming dates.

• Display the current date and time.

• Display today's date.

• Calculate and display the date after 7 days.

• Calculate and display the date after 30 days.

• Calculate and display the date after 1 month.

• Calculate and display the date after 1 year.

• Use DateTime methods.

PART 2: ARRAYS AND COLLECTIONS

6. Sports Performance Tracker

Store the running times of 10 athletes in an array.

• Accept running times from the user.

• Display all running times.

• Find the fastest time.

• Find the slowest time.

• Calculate the average running time.

7. Movie Watchlist

Create a movie watchlist using List<string>.

• Add movies to the watchlist.

• Remove a movie.

• Display all movies.

• Display the total number of movies.

• Display a movie using its index.

• Allow the user to add multiple movies.

8. Library Book Inventory

Create a book inventory using Dictionary<string, int>. Use the book title as the key and the available quantity as the value.

• Add a book and its quantity.

• Update the quantity of an existing book.

• Search for a book.

• Remove a book.

• Display all books and their quantities.

• Check whether a book exists using ContainsKey().

9. Daily Water Intake Tracker

Store daily water consumption for 7 days using List<double>.

• Accept water consumption for each day.

• Display all recorded values.

• Find the highest consumption.

• Find the lowest consumption.

• Calculate the average consumption.

• Add a new day's record.

• Remove a record.

10. Cinema Seat Arrangement

Create a cinema seating arrangement using a 2D array. Consider a cinema hall with 5 rows and 6 columns.

• Initially mark all seats as available.

• Display the seating arrangement.

• Allow users to book seats using row and column numbers.

• Mark booked seats as unavailable.

• Prevent booking an already-booked seat.

• Display the number of available seats.

PART 3: LINQ

11. Number Filtering System

Store 20 integers in a List<int>.

• Display all even numbers using Where().

• Display all odd numbers using Where().

• Display numbers greater than 50.

• Display numbers between 20 and 80.

• Sort numbers in ascending order using OrderBy().

12. Product Price Analyzer

Store the prices of 10 products in a List<double>.

• Display products costing more than ■500.

• Display products costing less than ■100.

• Sort product prices in ascending order.

• Create a new collection containing prices after applying a 10% discount using Select().

• Display the discounted prices.

13. Name Sorting Application

Accept 10 names and store them in a List<string>.

• Display names in alphabetical order.

• Display names starting with a particular character using Where().

• Display names containing a particular word or character.

• Create a new collection containing all names in uppercase using Select().

14. Inventory Stock Analyzer

Store item names and quantities in a Dictionary<string, int>.

• Add inventory items.

• Update item quantities.

• Display all inventory items.

• Display items with quantities below 10 using Where().

• Display items with quantities above 50.

• Sort items by quantity using OrderBy().

PART 4: MINI PROJECTS

15. Personal Expense Tracker

Create an expense tracker using List<double>.

• Add expenses.

• Remove expenses.

• Display all expenses.

• Display the total number of expenses.

• Calculate the total expenses using a user-defined method.

• Find the highest and lowest expenses.

• Calculate the average expenses.

• Use LINQ to display expenses above ■500.

• Sort expenses in ascending order.

16. Inventory Management System

Create an inventory management application using Dictionary<string, int>.

• Add new items.

• Update item quantities.

• Remove items.

• Search for an item.

• Display all available items.

• Display items that are out of stock.

• Display items with quantities below 5.

• Check item availability using ContainsKey().

17. Quiz Score Analyzer

Store quiz scores of multiple participants in a List<int>.

• Accept scores from the user.

• Display all scores.

• Find the highest score.

• Find the lowest score.

• Calculate the average score using a method.

• Display scores above the average using Where().

• Sort scores in ascending order.

• Create a new collection containing scores multiplied by 2 using Select().

18. Text Search Application

Create a program that accepts a sentence and a search word.

• Accept a sentence from the user.

• Remove unnecessary spaces using Trim().

• Convert the sentence into lowercase.

• Check whether the sentence contains the search word.

• Check whether the sentence starts with a given word.

• Check whether the sentence ends with a given word.

• Replace a particular word with another word.

• Display the modified sentence.

19. Event Registration Manager

Use List<string> to manage event registrations.

• Register participants.

• Cancel registrations.

• Display all registered participants.

• Display the total number of registrations.

• Display a participant using their index.

• Sort participant names alphabetically using OrderBy().

• Display names starting with a particular letter using Where().

20. Meter Reading Analyzer

Analyze a collection of meter readings using List<double>.

• Accept multiple meter readings.

• Store readings in a List.

• Create a separate method to calculate the minimum reading.

• Create a separate method to calculate the maximum reading.

• Create a separate method to calculate the average reading.

• Use LINQ to filter readings greater than 100 using Where().

• Use LINQ to sort readings in ascending order using OrderBy().

• Display all calculated results.